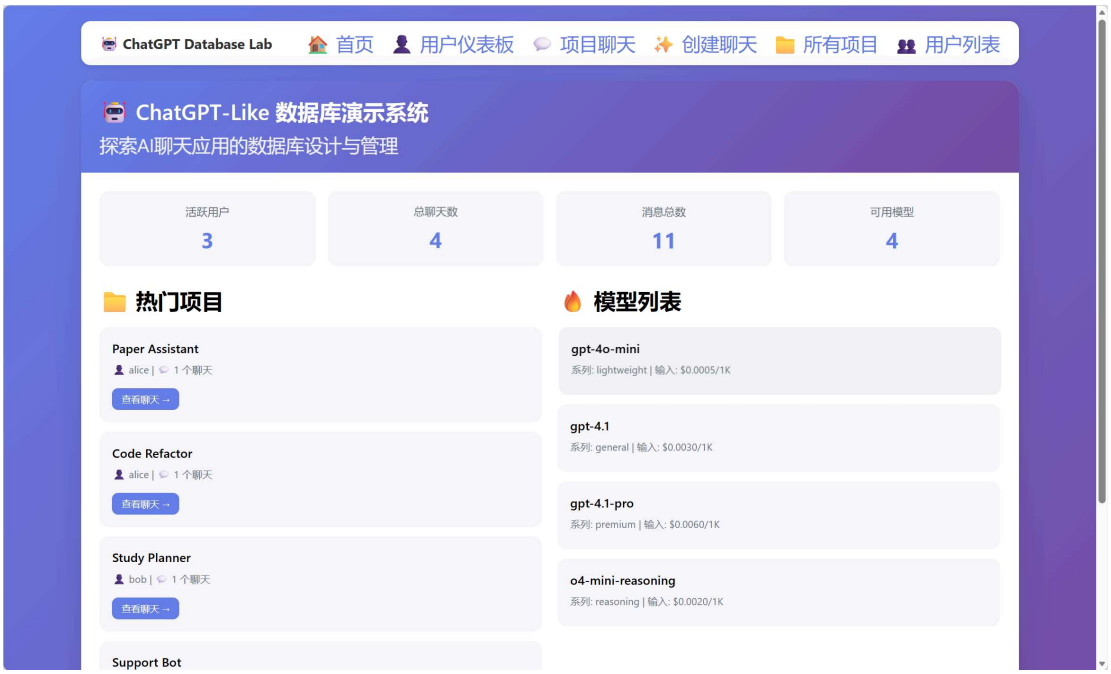
Lab 1: ChatGPT Database Application Design

组员： 毛川 2300013218

关睿轩 2300013238

芮思铭 2300013094



Scenario: Build a database for a ChatGPT-like app, focusing on:

- different model types
- user subscription tiers (`free / plus / pro`)
- token-based billing
- projects, chats, and context snapshots

This notebook covers all five required steps:

1. Business requirements
2. Database design + ER diagram
3. Create DB + seed data
4. SQL business operations + PySQLite CRUD
5. Flask web demo pages

Step 1: Business Requirements

1. Users can register and manage accounts.
2. Users can subscribe to `free` , `plus` , or `pro` plans.
3. Each plan controls which models can be used.

4. Multiple models exist with different input/output token prices.
5. Users organize work by projects.
6. Each project contains multiple chats.
7. Each chat has a default model and a system prompt.
8. Messages are stored with role (`user/assistant/system`) and token counts.
9. Context snapshots are stored for long conversations.
10. Usage records store input/output tokens and cost.
11. Wallet transactions store recharge/charge/refund and running balance.
12. The system supports common queries and CRUD operations.

Step 2: Database Design

2.1 Entities

- `users`
- `plans`
- `subscriptions`
- `models`
- `plan_model_access`
- `projects`
- `chats`
- `context_snapshots`
- `messages`
- `usage_records`
- `wallet_transactions`

The entries above are aligned one-to-one with the C-style structs below.

```
USERS {
    int user_id PK
    string username
    string email
    string created_at
}

PLANS {
    string plan_type PK
    float monthly_fee
    int monthly_quota_tokens
    string description
}

SUBSCRIPTIONS {
    int subscription_id PK
    int user_id FK
    string plan_type FK
    string start_date
    string end_date
    string status
}
```

```
MODELS {
    int model_id PK
    string model_name
    string family
    float input_price_per_1k
    float output_price_per_1k
    int max_context_tokens
}

PLAN_MODEL_ACCESS {
    string plan_type FK
    int model_id FK
    int can_use
}

PROJECTS {
    int project_id PK
    int user_id FK
    string project_name
    string description
    string created_at
}

CHATS {
    int chat_id PK
    int project_id FK
    int model_id FK
    string title
    string system_prompt
    int is_archived
    string created_at
}

CONTEXT_SNAPSHOTS {
    int snapshot_id PK
    int chat_id FK
    int snapshot_no
    string summary_text
    int token_count
    string created_at
}

MESSAGES {
    int message_id PK
    int chat_id FK
    string role
    string content
    int input_tokens
    int output_tokens
    string created_at
}

USAGE_RECORDS {
    int usage_id PK
    int user_id FK
```

```
    int chat_id FK
    int model_id FK
    int message_id FK
    int input_tokens
    int output_tokens
    float cost
    string billed_at
}

WALLET_TRANSACTIONS {
    int tx_id PK
    int user_id FK
    string tx_type
    float amount
    float balance_after
    string note
    string created_at
}
```

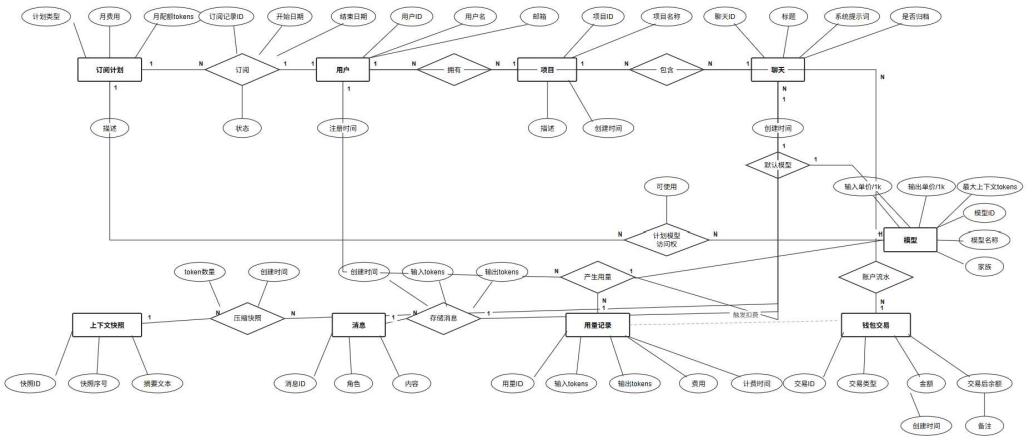
2.2 Core Relations (Natural-Language Description)

- `users` and `subscriptions` form a **one-to-many history relation**: one user may have multiple subscription records over time (upgrade, downgrade, renewal), but at a given time only one record should normally be `active`. This design keeps pricing history auditable.
- `plans` and `subscriptions` form a **one-to-many policy relation**: a plan (free/plus/pro) can be assigned to many users, while each subscription record points to exactly one plan. This decouples user behavior from plan definition changes.
- `plans` and `models` are linked by `plan_model_access` as a **many-to-many capability relation**: one plan can open multiple models, and one model can be available to multiple plans. The `can_use` flag allows temporary switches without deleting mappings.
- `users` and `projects` are **one-to-many ownership**: a project belongs to exactly one user, but a user can maintain many projects for different goals (study, coding, support automation, etc.).
- `projects` and `chats` are **one-to-many organization**: a project groups multiple chats under one theme, while each chat is scoped to one project for clean context management.
- `models` and `chats` are **one-to-many default-model assignment**: each chat chooses one default model, while the same model can serve many chats. This supports consistent behavior within a conversation.
- `chats` and `messages` are **one-to-many timeline storage**: a chat contains ordered messages with role and token counts; each message belongs to exactly one chat.
- `chats` and `context_snapshots` are **one-to-many compression checkpoints**: one chat can have multiple snapshots (`snapshot_no`) to preserve summarized memory as conversation length grows.

- `users`, `chats`, `models`, and optionally `messages` connect to `usage_records` as a **billing fact relation**: each usage row records who used which model in which chat, token consumption, and computed cost. This table is the analytical center for billing and cost reports.
- `users` and `wallet_transactions` are **one-to-many ledger relation**: every balance change (recharge/charge/refund) is stored as a transaction row; the latest row represents current balance, and all rows together provide a full financial trail.
- `usage_records` and `wallet_transactions` are a **business-level causal relation** (not strict FK): one usage event may trigger one charge transaction, enabling traceable cost-to-payment linkage in downstream reconciliation.

2.3 ER Diagram

Rendered ER image (saved locally):



In [1]:
```
import sqlite3
from pathlib import Path

DB_PATH = Path("chatgpt_lab.db")
print("Database file:", DB_PATH.resolve())
print("Tip: if Flask UI is running, stop it before re-initializing schema.")
```

```
Database file: D:\Courses\sixth_semester\Database\lab\lab1\chatgpt_lab.db
Tip: if Flask UI is running, stop it before re-initializing schema.
```

In [2]:
```
# Step 3: Create database schema
schema_sql = """
PRAGMA foreign_keys = OFF;

DROP TABLE IF EXISTS wallet_transactions;
DROP TABLE IF EXISTS usage_records;
DROP TABLE IF EXISTS messages;
DROP TABLE IF EXISTS context_snapshots;
DROP TABLE IF EXISTS chats;
DROP TABLE IF EXISTS projects;
DROP TABLE IF EXISTS plan_model_access;
DROP TABLE IF EXISTS models;
DROP TABLE IF EXISTS subscriptions;
```

```
DROP TABLE IF EXISTS plans;
DROP TABLE IF EXISTS users;

PRAGMA foreign_keys = ON;

CREATE TABLE users (
    user_id INTEGER PRIMARY KEY AUTOINCREMENT,
    username TEXT NOT NULL UNIQUE,
    email TEXT NOT NULL UNIQUE,
    created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE plans (
    plan_type TEXT PRIMARY KEY,
    monthly_fee REAL NOT NULL,
    monthly_quota_tokens INTEGER NOT NULL,
    description TEXT
);

CREATE TABLE subscriptions (
    subscription_id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id INTEGER NOT NULL,
    plan_type TEXT NOT NULL,
    start_date TEXT NOT NULL,
    end_date TEXT,
    status TEXT NOT NULL CHECK(status IN ('active', 'expired', 'cancelled')),
    FOREIGN KEY (user_id) REFERENCES users(user_id),
    FOREIGN KEY (plan_type) REFERENCES plans(plan_type)
);

CREATE TABLE models (
    model_id INTEGER PRIMARY KEY AUTOINCREMENT,
    model_name TEXT NOT NULL UNIQUE,
    family TEXT NOT NULL,
    input_price_per_1k REAL NOT NULL,
    output_price_per_1k REAL NOT NULL,
    max_context_tokens INTEGER NOT NULL
);

CREATE TABLE plan_model_access (
    plan_type TEXT NOT NULL,
    model_id INTEGER NOT NULL,
    can_use INTEGER NOT NULL DEFAULT 1 CHECK(can_use IN (0, 1)),
    PRIMARY KEY (plan_type, model_id),
    FOREIGN KEY (plan_type) REFERENCES plans(plan_type),
    FOREIGN KEY (model_id) REFERENCES models(model_id)
);

CREATE TABLE projects (
    project_id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id INTEGER NOT NULL,
    project_name TEXT NOT NULL,
    description TEXT,
    created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES users(user_id)
);

CREATE TABLE chats (
    chat_id INTEGER PRIMARY KEY AUTOINCREMENT,
    project_id INTEGER NOT NULL,
```

```
    model_id INTEGER NOT NULL,
    title TEXT NOT NULL,
    system_prompt TEXT,
    is_archived INTEGER NOT NULL DEFAULT 0 CHECK(is_archived IN (0, 1)),
    created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (project_id) REFERENCES projects(project_id) ON DELETE CASCADE,
    FOREIGN KEY (model_id) REFERENCES models(model_id)
);

CREATE TABLE context_snapshots (
    snapshot_id INTEGER PRIMARY KEY AUTOINCREMENT,
    chat_id INTEGER NOT NULL,
    snapshot_no INTEGER NOT NULL,
    summary_text TEXT NOT NULL,
    token_count INTEGER NOT NULL,
    created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (chat_id) REFERENCES chats(chat_id) ON DELETE CASCADE
);

CREATE TABLE messages (
    message_id INTEGER PRIMARY KEY AUTOINCREMENT,
    chat_id INTEGER NOT NULL,
    role TEXT NOT NULL CHECK(role IN ('system', 'user', 'assistant')),
    content TEXT NOT NULL,
    input_tokens INTEGER NOT NULL DEFAULT 0,
    output_tokens INTEGER NOT NULL DEFAULT 0,
    created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (chat_id) REFERENCES chats(chat_id) ON DELETE CASCADE
);

CREATE TABLE usage_records (
    usage_id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id INTEGER NOT NULL,
    chat_id INTEGER NOT NULL,
    model_id INTEGER NOT NULL,
    message_id INTEGER,
    input_tokens INTEGER NOT NULL,
    output_tokens INTEGER NOT NULL,
    cost REAL NOT NULL,
    billed_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES users(user_id),
    FOREIGN KEY (chat_id) REFERENCES chats(chat_id),
    FOREIGN KEY (model_id) REFERENCES models(model_id),
    FOREIGN KEY (message_id) REFERENCES messages(message_id) ON DELETE SET NULL
);

CREATE TABLE wallet_transactions (
    tx_id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id INTEGER NOT NULL,
    tx_type TEXT NOT NULL CHECK(tx_type IN ('recharge', 'charge', 'refund')),
    amount REAL NOT NULL,
    balance_after REAL NOT NULL,
    note TEXT,
    created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES users(user_id)
);

CREATE INDEX idx_messages_chat_time ON messages(chat_id, created_at);
CREATE INDEX idx_usage_user_time ON usage_records(user_id, billed_at);
"""
```

```
with sqlite3.connect(DB_PATH) as conn:
    conn.executescript(schema_sql)

print("Schema created")
```

```
Schema created
```

In [3]:

```
# Step 3: Seed sample data
with sqlite3.connect(DB_PATH) as conn:
    conn.execute("PRAGMA foreign_keys = ON;")

    conn.executemany(
        "INSERT INTO users(user_id, username, email, created_at) VALUES (?, ?, ?
        [
            (1, "alice", "alice@example.com", "2026-04-01 09:00:00"),
            (2, "bob", "bob@example.com", "2026-04-01 09:10:00"),
            (3, "charlie", "charlie@example.com", "2026-04-01 09:20:00"),
        ],
    )

    conn.executemany(
        "INSERT INTO plans(plan_type, monthly_fee, monthly_quota_tokens, descrip
        [
            ("free", 0, 100000, "Basic free plan"),
            ("plus", 20, 2000000, "Personal high-usage plan"),
            ("pro", 200, 20000000, "Team and enterprise plan"),
        ],
    )

    conn.executemany(
        "INSERT INTO subscriptions(subscription_id, user_id, plan_type, start_da
        [
            (1, 1, "plus", "2026-04-01", "2026-05-01", "active"),
            (2, 2, "free", "2026-04-01", None, "active"),
            (3, 3, "pro", "2026-04-01", "2026-05-01", "active"),
        ],
    )

    conn.executemany(
        "INSERT INTO models(model_id, model_name, family, input_price_per_1k, ou
        [
            (1, "gpt-4o-mini", "lightweight", 0.0005, 0.0015, 128000),
            (2, "gpt-4.1", "general", 0.0030, 0.0090, 128000),
            (3, "gpt-4.1-pro", "premium", 0.0060, 0.0180, 200000),
            (4, "o4-mini-reasoning", "reasoning", 0.0020, 0.0060, 200000),
        ],
    )

    conn.executemany(
        "INSERT INTO plan_model_access(plan_type, model_id, can_use) VALUES (?,
        [
            ("free", 1, 1),
            ("plus", 1, 1), ("plus", 2, 1), ("plus", 4, 1),
            ("pro", 1, 1), ("pro", 2, 1), ("pro", 3, 1), ("pro", 4, 1),
        ],
    )

    conn.executemany(
```

```
        "INSERT INTO projects(project_id, user_id, project_name, description, cr
        [
            (1, 1, "Paper Assistant", "Summarize LLM papers", "2026-04-01 10:00:
            (2, 1, "Code Refactor", "Refactor Flask app", "2026-04-01 10:30:00")
            (3, 2, "Study Planner", "Weekly study planning", "2026-04-01 11:00:0
            (4, 3, "Support Bot", "Generate customer support SOP", "2026-04-01 1
        ],
    )

    conn.executemany(
        "INSERT INTO chats(chat_id, project_id, model_id, title, system_prompt,
        [
            (1, 1, 2, "Literature Review Draft", "You are a rigorous academic as
            (2, 2, 4, "Refactor Login Module", "You are a senior backend enginee
            (3, 3, 1, "Daily Study Plan", "You are a learning planning assistant
            (4, 4, 3, "Support SOP Generator", "You are an enterprise support ex
        ],
    )

    conn.executemany(
        "INSERT INTO context_snapshots(snapshot_id, chat_id, snapshot_no, summar
        [
            (1, 1, 1, "User needs a summary of five papers with innovation highl
            (2, 2, 1, "User wants login logic split into service and validation
        ],
    )

    conn.executemany(
        "INSERT INTO messages(message_id, chat_id, role, content, input_tokens,
        [
            (1, 1, "user", "Summarize five LLM papers and highlight contribution
            (2, 1, "assistant", "I will summarize them by problem, method, and f
            (3, 2, "user", "Login module is messy. Propose layered refactor.", 8
            (4, 2, "assistant", "Use routing, service, repository layers with un
            (5, 3, "user", "Plan my database study for next week.", 40, 0, "2026
            (6, 3, "assistant", "Split into ER modeling, SQL drills, and interfa
            (7, 4, "user", "Give me a refund scenario support script.", 150, 0,
            (8, 4, "assistant", "Use four parts: empathy, verification, handling
            (9, 1, "user", "Also add limitations for each paper.", 70, 0, "2026-
            (10, 1, "assistant", "Added limits on scale, interpretability, and g
        ],
    )

    conn.executemany(
        "INSERT INTO usage_records(usage_id, user_id, chat_id, model_id, message
        [
            (1, 1, 1, 2, 2, 120, 420, 0.004140, "2026-04-02 09:01:10"),
            (2, 1, 2, 4, 4, 80, 260, 0.001720, "2026-04-02 09:31:12"),
            (3, 2, 3, 1, 6, 40, 150, 0.000245, "2026-04-02 10:01:07"),
            (4, 3, 4, 3, 8, 150, 500, 0.009900, "2026-04-02 10:31:20"),
            (5, 1, 1, 2, 10, 70, 200, 0.002010, "2026-04-02 09:05:10"),
        ],
    )

    conn.executemany(
        "INSERT INTO wallet_transactions(tx_id, user_id, tx_type, amount, balanc
        [
            (1, 1, "recharge", 20.0, 20.0, "Initial plus recharge", "2026-04-01
            (2, 1, "charge", -0.004140, 19.995860, "chat#1 usage#1", "2026-04-02
            (3, 1, "charge", -0.001720, 19.994140, "chat#2 usage#2", "2026-04-02
```

```
            (4, 1, "charge", -0.002010, 19.992130, "chat#1 usage#5", "2026-04-02
            (5, 2, "recharge", 5.0, 5.0, "Wallet recharge", "2026-04-01 09:15:00
            (6, 2, "charge", -0.000245, 4.999755, "chat#3 usage#3", "2026-04-02
            (7, 3, "recharge", 50.0, 50.0, "Initial pro recharge", "2026-04-01 0
            (8, 3, "charge", -0.009900, 49.990100, "chat#4 usage#4", "2026-04-02
        ],
    )

print("Seed data inserted")
```

```
Seed data inserted
```

In [4]:
```
# Quick check: row counts
with sqlite3.connect(DB_PATH) as conn:
    tables = [r[0] for r in conn.execute(
        "SELECT name FROM sqlite_master WHERE type='table' AND name NOT LIKE 'sq
    )]
    for t in tables:
        n = conn.execute(f"SELECT COUNT(*) FROM {t}").fetchone()[0]
        print(f"{t:<22} {n}")
```

```
chats                  4
context_snapshots      2
messages               10
models                 4
plan_model_access      8
plans                  3
projects               4
subscriptions          3
usage_records          5
users                  3
wallet_transactions    8
```

Step 4: SQL Business Operations (~10)

Each item maps to a concrete business function.

In [5]:
```python
def run_select(title, sql, params=()):
    print("\n" + "=" * 80)
    print(title)
    with sqlite3.connect(DB_PATH) as conn:
        conn.row_factory = sqlite3.Row
        rows = conn.execute(sql, params).fetchall()
    if not rows:
        print("(no rows)")
        return
    cols = rows[0].keys()
    print(" | ".join(cols))
    for r in rows:
        print(" | ".join(str(r[c]) for c in cols))


def run_dml(title, sql, params=()):
    with sqlite3.connect(DB_PATH) as conn:
        conn.execute("PRAGMA foreign_keys = ON;")
        cur = conn.execute(sql, params)
        conn.commit()
        print(f"{title} -> affected {cur.rowcount} row(s), lastrowid={cur.lastrc
        return cur.lastrowid
```

In [6]:
```python
# 1) models available to a user by plan
run_select(
    "1) Models available to alice",
    """
    SELECT u.username, s.plan_type, m.model_name, m.family
    FROM users u
    JOIN subscriptions s ON s.user_id = u.user_id AND s.status = 'active'
    JOIN plan_model_access pma ON pma.plan_type = s.plan_type AND pma.can_use =
    JOIN models m ON m.model_id = pma.model_id
    WHERE u.user_id = ?
    ORDER BY m.model_id;
    """,
    (1,),
)

# 2) chats under a project
run_select(
    "2) Chats under project #1",
    """
    SELECT p.project_name, c.chat_id, c.title, m.model_name, c.created_at
    FROM projects p
    JOIN chats c ON c.project_id = p.project_id
    JOIN models m ON m.model_id = c.model_id
    WHERE p.project_id = ?
    ORDER BY c.created_at;
    """,
    (1,),
)

# 3) recent messages in a chat
run_select(
    "3) Recent messages in chat #1",
    """
    SELECT message_id, role, content, input_tokens, output_tokens, created_at
    FROM messages
    WHERE chat_id = ?
```

```
    ORDER BY created_at DESC
    LIMIT 6;
    """,
    (1,),
)

# 4) monthly token/cost summary per user
run_select(
    "4) Monthly summary for 2026-04",
    """
    SELECT u.username,
           SUM(ur.input_tokens)  AS total_input_tokens,
           SUM(ur.output_tokens) AS total_output_tokens,
           ROUND(SUM(ur.cost), 6) AS total_cost
    FROM usage_records ur
    JOIN users u ON u.user_id = ur.user_id
    WHERE strftime('%Y-%m', ur.billed_at) = '2026-04'
    GROUP BY u.user_id
    ORDER BY total_cost DESC;
    """,
)

# 5) top expensive chats
run_select(
    "5) Top 3 expensive chats",
    """
    SELECT c.chat_id, c.title, u.username, ROUND(SUM(ur.cost), 6) AS chat_cost
    FROM usage_records ur
    JOIN chats c ON c.chat_id = ur.chat_id
    JOIN projects p ON p.project_id = c.project_id
    JOIN users u ON u.user_id = p.user_id
    GROUP BY c.chat_id
    ORDER BY chat_cost DESC
    LIMIT 3;
    """,
)

# 6) insert a new user message
new_user_msg_id = run_dml(
    "6) Insert new user message",
    """
    INSERT INTO messages(chat_id, role, content, input_tokens, output_tokens, cr
    VALUES (?, 'user', ?, ?, 0, ?);
    """,
    (2, "Please split the refactor into iteration 1 and 2.", 35, "2026-04-05 10:
)

# 7) insert assistant reply + usage record with dynamic cost calc
new_assistant_msg_id = run_dml(
    "7-1) Insert assistant message",
    """
    INSERT INTO messages(chat_id, role, content, input_tokens, output_tokens, cr
    VALUES (?, 'assistant', ?, ?, ?, ?);
    """,
    (2, "Iteration 1: auth and exception handling. Iteration 2: cache and audit.
)

run_dml(
    "7-2) Insert usage record",
    """
```

```
    INSERT INTO usage_records(user_id, chat_id, model_id, message_id, input_toke
    SELECT p.user_id, c.chat_id, c.model_id, ?, ?, ?,
           ROUND(? * m.input_price_per_1k / 1000.0 + ? * m.output_price_per_1k /
           ?
    FROM chats c
    JOIN projects p ON p.project_id = c.project_id
    JOIN models m ON m.model_id = c.model_id
    WHERE c.chat_id = ?;
    """,
    (new_assistant_msg_id, 35, 120, 35, 120, "2026-04-05 10:05:20", 2),
)

# 8) update chat title
run_dml(
    "8) Update chat title",
    "UPDATE chats SET title = ? WHERE chat_id = ?;",
    ("Refactor Login Module (Two Iterations)", 2),
)

# 9) delete mistaken message
run_dml(
    "9) Delete mistaken message",
    "DELETE FROM messages WHERE message_id = ?;",
    (new_user_msg_id,),
)

# 10) current wallet balance per user
run_select(
    "10) Current wallet balance",
    """
    SELECT u.user_id, u.username, wt.balance_after AS current_balance
    FROM users u
    JOIN wallet_transactions wt ON wt.tx_id = (
        SELECT tx2.tx_id
        FROM wallet_transactions tx2
        WHERE tx2.user_id = u.user_id
        ORDER BY tx2.created_at DESC, tx2.tx_id DESC
        LIMIT 1
    )
    ORDER BY u.user_id;
    """,
)
```

```
================================================================================
1) Models available to alice
username | plan_type | model_name | family
alice | plus | gpt-4o-mini | lightweight
alice | plus | gpt-4.1 | general
alice | plus | o4-mini-reasoning | reasoning

================================================================================
2) Chats under project #1
project_name | chat_id | title | model_name | created_at
Paper Assistant | 1 | Literature Review Draft | gpt-4.1 | 2026-04-02 09:00:00

================================================================================
3) Recent messages in chat #1
message_id | role | content | input_tokens | output_tokens | created_at
10 | assistant | Added limits on scale, interpretability, and generalization. | 7
0 | 200 | 2026-04-02 09:05:10
9 | user | Also add limitations for each paper. | 70 | 0 | 2026-04-02 09:05:00
2 | assistant | I will summarize them by problem, method, and findings. | 120 | 4
20 | 2026-04-02 09:01:10
1 | user | Summarize five LLM papers and highlight contributions. | 120 | 0 | 202
6-04-02 09:01:00

================================================================================
4) Monthly summary for 2026-04
username | total_input_tokens | total_output_tokens | total_cost
charlie | 150 | 500 | 0.0099
alice | 270 | 880 | 0.00787
bob | 40 | 150 | 0.000245

================================================================================
5) Top 3 expensive chats
chat_id | title | username | chat_cost
4 | Support SOP Generator | charlie | 0.0099
1 | Literature Review Draft | alice | 0.00615
2 | Refactor Login Module | alice | 0.00172
6) Insert new user message -> affected 1 row(s), lastrowid=11
7-1) Insert assistant message -> affected 1 row(s), lastrowid=12
7-2) Insert usage record -> affected 1 row(s), lastrowid=6
8) Update chat title -> affected 1 row(s), lastrowid=0
9) Delete mistaken message -> affected 1 row(s), lastrowid=0

================================================================================
10) Current wallet balance
user_id | username | current_balance
1 | alice | 19.99414
2 | bob | 4.999755
3 | charlie | 49.9901
```

Step 4 (extra): PySQLite CRUD API

Required CRUD operations implemented with Python DB interface.

In [7]:

```python
def create_chat(project_id, model_id, title, system_prompt):
    with sqlite3.connect(DB_PATH) as conn:
        conn.execute("PRAGMA foreign_keys = ON;")
        cur = conn.execute(
            """
```

```
            INSERT INTO chats(project_id, model_id, title, system_prompt, is_arc
            VALUES (?, ?, ?, ?, 0, datetime('now'));
            """,
            (project_id, model_id, title, system_prompt),
        )
        conn.commit()
        return cur.lastrowid


def create_message(chat_id, role, content, input_tokens=0, output_tokens=0):
    with sqlite3.connect(DB_PATH) as conn:
        conn.execute("PRAGMA foreign_keys = ON;")
        cur = conn.execute(
            """
            INSERT INTO messages(chat_id, role, content, input_tokens, output_to
            VALUES (?, ?, ?, ?, ?, datetime('now'));
            """,
            (chat_id, role, content, input_tokens, output_tokens),
        )
        conn.commit()
        return cur.lastrowid


def read_chat(chat_id):
    with sqlite3.connect(DB_PATH) as conn:
        conn.row_factory = sqlite3.Row
        chat = conn.execute(
            """
            SELECT c.chat_id, c.title, c.system_prompt, m.model_name, p.project_
            FROM chats c
            JOIN models m ON m.model_id = c.model_id
            JOIN projects p ON p.project_id = c.project_id
            WHERE c.chat_id = ?;
            """,
            (chat_id,),
        ).fetchone()
        if not chat:
            return None
        msgs = conn.execute(
            """
            SELECT message_id, role, content, input_tokens, output_tokens, creat
            FROM messages
            WHERE chat_id = ?
            ORDER BY message_id;
            """,
            (chat_id,),
        ).fetchall()
        return {"chat": dict(chat), "messages": [dict(m) for m in msgs]}


def update_chat_title(chat_id, new_title):
    with sqlite3.connect(DB_PATH) as conn:
        cur = conn.execute("UPDATE chats SET title = ? WHERE chat_id = ?;", (new
        conn.commit()
        return cur.rowcount


def delete_chat(chat_id):
    with sqlite3.connect(DB_PATH) as conn:
        conn.execute("PRAGMA foreign_keys = ON;")
```

```
        cur = conn.execute("DELETE FROM chats WHERE chat_id = ?;", (chat_id,))
        conn.commit()
        return cur.rowcount


# CRUD demo
sample_chat_id = create_chat(1, 1, "CRUD Demo Chat", "You are a database TA.")
create_message(sample_chat_id, "user", "Please give me a database review outline
create_message(sample_chat_id, "assistant", "Review ER, normalization, SQL, and

print("Create + Read:")
print(read_chat(sample_chat_id))

print("\nUpdate:")
update_chat_title(sample_chat_id, "CRUD Demo Chat (Renamed)")
print(read_chat(sample_chat_id)["chat"])

print("\nDelete:")
delete_chat(sample_chat_id)
print("After delete:", read_chat(sample_chat_id))
```

```
Create + Read:
{'chat': {'chat_id': 5, 'title': 'CRUD Demo Chat', 'system_prompt': 'You are a da
tabase TA.', 'model_name': 'gpt-4o-mini', 'project_name': 'Paper Assistant'}, 'me
ssages': [{'message_id': 13, 'role': 'user', 'content': 'Please give me a databas
e review outline.', 'input_tokens': 30, 'output_tokens': 0, 'created_at': '2026-0
4-29 11:00:38'}, {'message_id': 14, 'role': 'assistant', 'content': 'Review ER, n
ormalization, SQL, and transactions.', 'input_tokens': 30, 'output_tokens': 110,
'created_at': '2026-04-29 11:00:38'}]}

Update:
{'chat_id': 5, 'title': 'CRUD Demo Chat (Renamed)', 'system_prompt': 'You are a d
atabase TA.', 'model_name': 'gpt-4o-mini', 'project_name': 'Paper Assistant'}

Delete:
After delete: None
```

Step 5: Flask Web Demo

The following minimal Flask app provides:

1. User dashboard (plan, balance, monthly token/cost)
2. Project chat list
3. Chat message history
4. Create chat form

In [8]:
```
from flask import Flask, request, render_template_string, redirect, url_for, fla

app = Flask(__name__)
app.secret_key = 'supersecretkey'  # For flash messages

DB_PATH = Path("chatgpt_lab.db")

def fetch_all(sql, params=()):
    with sqlite3.connect(DB_PATH) as conn:
        conn.row_factory = sqlite3.Row
        rows = conn.execute(sql, params).fetchall()
    return [dict(r) for r in rows]
```

```
def fetch_one(sql, params=()):
    with sqlite3.connect(DB_PATH) as conn:
        conn.row_factory = sqlite3.Row
        row = conn.execute(sql, params).fetchone()
    return dict(row) if row else None

def execute(sql, params=()):
    with sqlite3.connect(DB_PATH) as conn:
        conn.execute("PRAGMA foreign_keys = ON;")
        cur = conn.execute(sql, params)
        conn.commit()
        return cur.lastrowid

# 全局CSS样式
BASE_HTML_TEMPLATE = """
<!DOCTYPE html>
<html lang="zh-CN">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>ChatGPT-Like App 数据库演示系统</title>
    <style>
        * {
            margin: 0;
            padding: 0;
            box-sizing: border-box;
        }

        body {
            font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', 'PingFan
            background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
            min-height: 100vh;
            padding: 20px;
        }

        .container {
            max-width: 1200px;
            margin: 0 auto;
        }

        .card {
            background: white;
            border-radius: 16px;
            box-shadow: 0 10px 40px rgba(0,0,0,0.1);
            overflow: hidden;
            margin-bottom: 20px;
        }

        .card-header {
            background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
            color: white;
            padding: 20px 24px;
        }

        .card-header h1, .card-header h2 {
            margin: 0;
            font-weight: 600;
        }
```

```
        .card-header h1 {
            font-size: 28px;
        }

        .card-header h2 {
            font-size: 22px;
        }

        .card-body {
            padding: 24px;
        }

        .nav-bar {
            background: white;
            border-radius: 12px;
            box-shadow: 0 2px 8px rgba(0,0,0,0.1);
            padding: 12px 24px;
            margin-bottom: 20px;
            display: flex;
            gap: 20px;
            align-items: center;
            flex-wrap: wrap;
        }

        .nav-bar a {
            color: #667eea;
            text-decoration: none;
            font-weight: 500;
            transition: color 0.3s;
        }

        .nav-bar a:hover {
            color: #764ba2;
            text-decoration: underline;
        }

        .nav-bar .brand {
            font-size: 18px;
            font-weight: bold;
            color: #333;
            margin-right: auto;
        }

        .stats-grid {
            display: grid;
            grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
            gap: 16px;
            margin-bottom: 24px;
        }

        .stat-card {
            background: #f8f9fa;
            border-radius: 12px;
            padding: 16px;
            text-align: center;
            transition: transform 0.2s;
        }

        .stat-card:hover {
            transform: translateY(-2px);
```

```
        }

        .stat-label {
            font-size: 14px;
            color: #6c757d;
            margin-bottom: 8px;
        }

        .stat-value {
            font-size: 28px;
            font-weight: bold;
            color: #667eea;
        }

        .info-row {
            display: flex;
            padding: 12px 0;
            border-bottom: 1px solid #e9ecef;
        }

        .info-label {
            width: 120px;
            font-weight: 600;
            color: #495057;
        }

        .info-value {
            flex: 1;
            color: #212529;
        }

        table {
            width: 100%;
            border-collapse: collapse;
        }

        table thead {
            background: #f8f9fa;
        }

        table th, table td {
            padding: 12px;
            text-align: left;
            border-bottom: 1px solid #e9ecef;
        }

        table th {
            font-weight: 600;
            color: #495057;
        }

        table tr:hover {
            background: #f8f9fa;
        }

        .btn {
            display: inline-block;
            padding: 10px 20px;
            border-radius: 8px;
            font-size: 14px;
```

```
            font-weight: 500;
            text-decoration: none;
            transition: all 0.3s;
            cursor: pointer;
            border: none;
            background: #667eea;
            color: white;
        }

        .btn:hover {
            background: #764ba2;
            transform: translateY(-1px);
            box-shadow: 0 4px 12px rgba(102,126,234,0.4);
        }

        .btn-secondary {
            background: #6c757d;
        }

        .btn-secondary:hover {
            background: #5a6268;
        }

        .btn-outline {
            background: transparent;
            border: 2px solid #667eea;
            color: #667eea;
        }

        .btn-outline:hover {
            background: #667eea;
            color: white;
        }

        .form-group {
            margin-bottom: 20px;
        }

        .form-group label {
            display: block;
            margin-bottom: 8px;
            font-weight: 500;
            color: #495057;
        }

        .form-group input, .form-group textarea, .form-group select {
            width: 100%;
            padding: 10px 12px;
            border: 1px solid #ddd;
            border-radius: 8px;
            font-size: 14px;
            transition: border-color 0.3s;
        }

        .form-group input:focus, .form-group textarea:focus, .form-group select:
            outline: none;
            border-color: #667eea;
            box-shadow: 0 0 0 3px rgba(102,126,234,0.1);
        }
```

```
        .message-list {
            max-height: 500px;
            overflow-y: auto;
        }

        .message-item {
            padding: 16px;
            margin-bottom: 12px;
            border-radius: 12px;
            background: #f8f9fa;
        }

        .message-item.user {
            background: linear-gradient(135deg, #e3f2fd 0%, #bbdef5 100%);
            border-left: 4px solid #2196f3;
        }

        .message-item.assistant {
            background: linear-gradient(135deg, #f3e5f5 0%, #e1bee7 100%);
            border-left: 4px solid #9c27b0;
        }

        .message-item .role {
            font-weight: bold;
            margin-bottom: 8px;
            font-size: 14px;
            color: #666;
        }

        .message-item .content {
            font-size: 15px;
            line-height: 1.5;
            color: #212529;
        }

        .message-item .time {
            font-size: 11px;
            color: #999;
            margin-top: 8px;
        }

        .badge {
            display: inline-block;
            padding: 4px 10px;
            border-radius: 20px;
            font-size: 12px;
            font-weight: 500;
        }

        .badge-primary {
            background: #e3f2fd;
            color: #1976d2;
        }

        .badge-success {
            background: #e8f5e9;
            color: #388e3c;
        }

        .badge-warning {
```

```
            background: #fff3e0;
            color: #f57c00;
        }

        .project-list {
            display: flex;
            flex-direction: column;
            gap: 12px;
        }

        .project-item, .chat-item {
            background: #f8f9fa;
            border-radius: 12px;
            padding: 16px;
            transition: all 0.3s;
        }

        .project-item:hover, .chat-item:hover {
            background: #e9ecef;
            transform: translateX(4px);
        }

        .project-title, .chat-title {
            font-weight: 600;
            font-size: 16px;
            margin-bottom: 8px;
        }

        .project-meta, .chat-meta {
            font-size: 13px;
            color: #6c757d;
            margin-bottom: 8px;
        }

        .alert {
            padding: 12px 16px;
            border-radius: 8px;
            margin-bottom: 20px;
        }

        .alert-success {
            background: #d4edda;
            border: 1px solid #c3e6cb;
            color: #155724;
        }

        .footer {
            text-align: center;
            padding: 20px;
            color: rgba(255,255,255,0.7);
            font-size: 14px;
        }

        @media (max-width: 768px) {
            .stats-grid {
                grid-template-columns: repeat(2, 1fr);
            }
            .info-row {
                flex-direction: column;
            }
```

```
            .info-label {
                margin-bottom: 4px;
            }
        }
    </style>
</head>
<body>
    <div class="container">
        <div class="nav-bar">
            <span class="brand">🤖 ChatGPT Database Lab</span>
            <a href="/">🏠 首页</a>
            <a href="/user/1/dashboard">👤 用户仪表板</a>
            <a href="/project/1/chats">💬 项目聊天</a>
            <a href="/chat/create">✨ 创建聊天</a>
            <a href="/projects">📁 所有项目</a>
            <a href="/users">👥 用户列表</a>
        </div>

        {% block content %}{% endblock %}

        <div class="footer">
            ChatGPT Database Application Demo | Built with Flask & SQLite
        </div>
    </div>
</body>
</html>
"""

@app.route("/")
def home():
    html = BASE_HTML_TEMPLATE.replace("{% block content %}{% endblock %}", """
    <div class="card">
        <div class="card-header">
            <h1>🤖 ChatGPT-Like 数据库演示系统</h1>
            <p style="margin-top: 8px; opacity: 0.9;">探索AI聊天应用的数据库设计
        </div>
        <div class="card-body">
            <div class="stats-grid">
                <div class="stat-card">
                    <div class="stat-label">活跃用户</div>
                    <div class="stat-value">{{ users_count }}</div>
                </div>
                <div class="stat-card">
                    <div class="stat-label">总聊天数</div>
                    <div class="stat-value">{{ chats_count }}</div>
                </div>
                <div class="stat-card">
                    <div class="stat-label">消息总数</div>
                    <div class="stat-value">{{ messages_count }}</div>
                </div>
                <div class="stat-card">
                    <div class="stat-label">可用模型</div>
                    <div class="stat-value">{{ models_count }}</div>
                </div>
            </div>

            <div style="display: grid; grid-template-columns: repeat(auto-fit, m
                <div>
                    <h3 style="margin-bottom: 16px;">📁 热门项目</h3>
                    <div class="project-list">
```

```
                    {% for proj in projects %}
                    <div class="project-item">
                        <div class="project-title">{{ proj.project_name }}</
                        <div class="project-meta">👤 {{ proj.username }} |
                        <a href="/project/{{ proj.project_id }}/chats" class
                    </div>
                    {% endfor %}
                </div>
            </div>
            <div>
                <h3 style="margin-bottom: 16px;">🔥 模型列表</h3>
                <div class="project-list">
                    {% for model in models %}
                    <div class="project-item">
                        <div class="project-title">{{ model.model_name }}</d
                        <div class="project-meta">系列: {{ model.family }} |
                    </div>
                    {% endfor %}
                </div>
            </div>
        </div>

        <div style="margin-top: 24px; text-align: center;">
            <a href="/user/1/dashboard" class="btn">🚀 开始使用 →</a>
        </div>
    </div>
</div>
""")

    # Get stats
    users_count = fetch_one("SELECT COUNT(*) as cnt FROM users")['cnt']
    chats_count = fetch_one("SELECT COUNT(*) as cnt FROM chats")['cnt']
    messages_count = fetch_one("SELECT COUNT(*) as cnt FROM messages")['cnt']
    models_count = fetch_one("SELECT COUNT(*) as cnt FROM models")['cnt']
    projects = fetch_all("""
        SELECT p.project_id, p.project_name, u.username, COUNT(c.chat_id) as cha
        FROM projects p
        JOIN users u ON u.user_id = p.user_id
        LEFT JOIN chats c ON c.project_id = p.project_id
        GROUP BY p.project_id
        LIMIT 6
    """)
    models = fetch_all("SELECT model_name, family, input_price_per_1k FROM model

    return render_template_string(html, users_count=users_count, chats_count=cha
                                  messages_count=messages_count, models_count=m
                                  projects=projects, models=models)

@app.route("/user/<int:user_id>/dashboard")
def user_dashboard(user_id):
    profile = fetch_one(
        """SELECT u.user_id, u.username, u.email, u.created_at, s.plan_type
        FROM users u
        JOIN subscriptions s ON s.user_id = u.user_id AND s.status = 'active'
        WHERE u.user_id = ?;""",
        (user_id,),
    )
    if not profile:
        return "User not found", 404
```

```
    balance = fetch_one(
        """SELECT balance_after
        FROM wallet_transactions
        WHERE user_id = ?
        ORDER BY created_at DESC, tx_id DESC
        LIMIT 1;""",
        (user_id,),
    )

    monthly = fetch_one(
        """SELECT IFNULL(SUM(input_tokens), 0) AS in_tokens,
                  IFNULL(SUM(output_tokens), 0) AS out_tokens,
                  IFNULL(ROUND(SUM(cost), 6), 0) AS cost
        FROM usage_records
        WHERE user_id = ? AND strftime('%Y-%m', billed_at) = '2026-04';""",
        (user_id,),
    )

    # Get recent chats
    recent_chats = fetch_all("""
        SELECT c.chat_id, c.title, m.model_name, c.created_at
        FROM chats c
        JOIN projects p ON p.project_id = c.project_id
        JOIN models m ON m.model_id = c.model_id
        WHERE p.user_id = ?
        ORDER BY c.created_at DESC
        LIMIT 5
    """, (user_id,))

    html = BASE_HTML_TEMPLATE.replace("{% block content %}{% endblock %}", """
    <div class="card">
        <div class="card-header">
            <h2>👤 {{ p.username }} 的仪表板</h2>
        </div>
        <div class="card-body">
            <div class="stats-grid">
                <div class="stat-card">
                    <div class="stat-label">当前套餐</div>
                    <div class="stat-value">{{ p.plan_type | upper }}</div>
                </div>
                <div class="stat-card">
                    <div class="stat-label">账户余额</div>
                    <div class="stat-value">${{ "%.2f"|format(balance) }}</div>
                </div>
                <div class="stat-card">
                    <div class="stat-label">本月消耗</div>
                    <div class="stat-value">${{ "%.4f"|format(m.cost) }}</div>
                </div>
                <div class="stat-card">
                    <div class="stat-label">Token使用</div>
                    <div class="stat-value">{{ m.in_tokens + m.out_tokens }}</di
                </div>
            </div>

            <div class="info-row">
                <div class="info-label">用户ID</div>
                <div class="info-value">{{ p.user_id }}</div>
            </div>
            <div class="info-row">
                <div class="info-label">邮箱</div>
```

```
                <div class="info-value">{{ p.email }}</div>
            </div>
            <div class="info-row">
                <div class="info-label">注册时间</div>
                <div class="info-value">{{ p.created_at }}</div>
            </div>
            <div class="info-row">
                <div class="info-label">输入Token</div>
                <div class="info-value">{{ m.in_tokens }}</div>
            </div>
            <div class="info-row">
                <div class="info-label">输出Token</div>
                <div class="info-value">{{ m.out_tokens }}</div>
            </div>

            {% if recent_chats %}
            <h3 style="margin: 24px 0 16px 0;">📝 最近聊天记录</h3>
            <div class="project-list">
                {% for chat in recent_chats %}
                <div class="chat-item">
                    <div class="chat-title">{{ chat.title }}</div>
                    <div class="chat-meta">模型: {{ chat.model_name }} | 创建: {
                    <a href="/chat/{{ chat.chat_id }}/messages" class="btn" styl
                </div>
                {% endfor %}
            </div>
            {% endif %}

            <div style="margin-top: 24px;">
                <a href="/" class="btn btn-secondary">← 返回首页</a>
                <a href="/projects" class="btn">📁 查看我的项目</a>
            </div>
        </div>
    </div>
    """)

    return render_template_string(html, p=profile, balance=balance["balance_afte
                                  m=monthly, recent_chats=recent_chats)

@app.route("/project/<int:project_id>/chats")
def project_chats(project_id):
    project = fetch_one("SELECT project_name FROM projects WHERE project_id = ?"
    rows = fetch_all(
        """SELECT c.chat_id, c.title, c.system_prompt, m.model_name, c.created_a
                  (SELECT COUNT(*) FROM messages WHERE chat_id = c.chat_id) as m
        FROM chats c
        JOIN models m ON m.model_id = c.model_id
        WHERE c.project_id = ?
        ORDER BY c.created_at DESC;""",
        (project_id,),
    )

    html = BASE_HTML_TEMPLATE.replace("{% block content %}{% endblock %}", """
    <div class="card">
        <div class="card-header">
            <h2>📁 {{ project_name }} - 聊天列表</h2>
            <p style="margin-top: 8px;">共 {{ rows|length }} 个对话</p>
        </div>
        <div class="card-body">
            {% if rows %}
```

```
                <div class="project-list">
                {% for r in rows %}
                    <div class="chat-item">
                        <div class="chat-title">
                            💬 {{ r.title }}
                            <span class="badge badge-primary" style="float: righ
                        </div>
                        <div class="chat-meta">
                            模型: <span class="badge badge-success">{{ r.model_n
                            创建时间: {{ r.created_at }}
                        </div>
                        {% if r.system_prompt %}
                        <div class="chat-meta">系统提示: {{ r.system_prompt[:80]
                        {% endif %}
                        <div style="margin-top: 12px;">
                            <a href="/chat/{{ r.chat_id }}/messages" class="btn"
                        </div>
                    </div>
                {% endfor %}
                </div>
            {% else %}
                <div class="alert" style="background: #fff3cd; color: #856404;">
                    📭 暂无聊天记录，<a href="/chat/create">点击创建新聊天</a>
                </div>
            {% endif %}

            <div style="margin-top: 24px;">
                <a href="/" class="btn btn-secondary">← 返回首页</a>
                <a href="/chat/create" class="btn">✨ 创建新聊天</a>
            </div>
        </div>
    </div>
    """)

    return render_template_string(html, project_name=project['project_name'] if

@app.route("/chat/<int:chat_id>/messages")
def chat_messages(chat_id):
    chat_info = fetch_one(
        """SELECT c.title, c.system_prompt, m.model_name, p.project_name
        FROM chats c
        JOIN models m ON m.model_id = c.model_id
        JOIN projects p ON p.project_id = c.project_id
        WHERE c.chat_id = ?;""",
        (chat_id,),
    )

    rows = fetch_all(
        """SELECT message_id, role, content, input_tokens, output_tokens, create
        FROM messages
        WHERE chat_id = ?
        ORDER BY message_id;""",
        (chat_id,),
    )

    html = BASE_HTML_TEMPLATE.replace("{% block content %}{% endblock %}", """
    <div class="card">
        <div class="card-header">
            <h2>💬 {{ chat_info.title if chat_info else '聊天' }}</h2>
            <p style="margin-top: 8px;">
```

```
                项目: {{ chat_info.project_name if chat_info else '未知' }} |
                模型: {{ chat_info.model_name if chat_info else '未知' }}
            </p>
        </div>
        <div class="card-body">
            {% if chat_info and chat_info.system_prompt %}
            <div class="alert" style="background: #e3f2fd; margin-bottom: 20px;"
                <strong>⚙️ 系统提示词:</strong> {{ chat_info.system_prompt }}
            </div>
            {% endif %}

            <div class="message-list">
                {% for r in rows %}
                <div class="message-item {{ r.role }}">
                    <div class="role">
                        {% if r.role == 'user' %}👤 用户{% elif r.role == 'assi
                        {% if r.input_tokens > 0 or r.output_tokens > 0 %}
                        <span class="badge" style="float: right; background: #e9
                            Tokens: {{ r.input_tokens + r.output_tokens }}
                        </span>
                        {% endif %}
                    </div>
                    <div class="content">{{ r.content }}</div>
                    <div class="time">{{ r.created_at }}</div>
                </div>
                {% endfor %}
            </div>

            {% if not rows %}
            <div class="alert" style="background: #fff3cd;">
                📭 暂无消息
            </div>
            {% endif %}

            <div style="margin-top: 24px;">
                <a href="javascript:history.back()" class="btn btn-secondary">←
                <a href="/" class="btn">🏠 首页</a>
            </div>
        </div>
    </div>
    """)

    return render_template_string(html, chat_info=chat_info, rows=rows)

@app.route("/chat/create", methods=["GET", "POST"])
def create_chat_page():
    if request.method == "POST":
        project_id = int(request.form["project_id"])
        model_id = int(request.form["model_id"])
        title = request.form["title"]
        system_prompt = request.form["system_prompt"]
        chat_id = execute(
            """INSERT INTO chats(project_id, model_id, title, system_prompt, is_
            VALUES (?, ?, ?, ?, 0, datetime('now'));""",
            (project_id, model_id, title, system_prompt),
        )
        return redirect(url_for("chat_messages", chat_id=chat_id))

    projects = fetch_all("SELECT project_id, project_name FROM projects ORDER BY
    models = fetch_all("SELECT model_id, model_name, family FROM models ORDER BY
```

```
    html = BASE_HTML_TEMPLATE.replace("{% block content %}{% endblock %}", """
    <div class="card">
        <div class="card-header">
            <h2>✨ 创建新聊天</h2>
            <p>选择一个项目和模型开始新的对话</p>
        </div>
        <div class="card-body">
            <form method="post">
                <div class="form-group">
                    <label>📁 选择项目</label>
                    <select name="project_id" required>
                        {% for p in projects %}
                        <option value="{{ p.project_id }}">{{ p.project_name }}<
                        {% endfor %}
                    </select>
                </div>

                <div class="form-group">
                    <label>🧠 选择模型</label>
                    <select name="model_id" required>
                        {% for m in models %}
                        <option value="{{ m.model_id }}">{{ m.model_name }} ({{
                        {% endfor %}
                    </select>
                </div>

                <div class="form-group">
                    <label>📝 聊天标题</label>
                    <input type="text" name="title" placeholder="输入聊天标题...
                </div>

                <div class="form-group">
                    <label>⚙️ 系统提示词 (可选)</label>
                    <textarea name="system_prompt" rows="3" placeholder="例如: 你
                </div>

                <div style="display: flex; gap: 12px;">
                    <button type="submit" class="btn">✨ 创建聊天</button>
                    <a href="/" class="btn btn-secondary">取消</a>
                </div>
            </form>
        </div>
    </div>
    """)

    return render_template_string(html, projects=projects, models=models)

@app.route("/projects")
def all_projects():
    projects = fetch_all("""
        SELECT p.project_id, p.project_name, p.description, p.created_at, u.user
               COUNT(DISTINCT c.chat_id) as chat_count,
               COUNT(DISTINCT m.message_id) as msg_count
        FROM projects p
        JOIN users u ON u.user_id = p.user_id
        LEFT JOIN chats c ON c.project_id = p.project_id
        LEFT JOIN messages m ON m.chat_id = c.chat_id
        GROUP BY p.project_id
        ORDER BY p.created_at DESC
```

```
    """)

    html = BASE_HTML_TEMPLATE.replace("{% block content %}{% endblock %}", """
    <div class="card">
        <div class="card-header">
            <h2>📁 所有项目</h2>
            <p>浏览所有用户创建的项目</p>
        </div>
        <div class="card-body">
            <div class="project-list">
                {% for p in projects %}
                <div class="project-item">
                    <div class="project-title">{{ p.project_name }}</div>
                    <div class="project-meta">
                        创建者: {{ p.username }} |
                        创建: {{ p.created_at }} |
                        聊天数: {{ p.chat_count }} |
                        消息数: {{ p.msg_count }}
                    </div>
                    {% if p.description %}
                    <div class="project-meta">描述: {{ p.description }}</div>
                    {% endif %}
                    <div style="margin-top: 12px;">
                        <a href="/project/{{ p.project_id }}/chats" class="btn"
                    </div>
                </div>
                {% endfor %}
            </div>
            <div style="margin-top: 24px;">
                <a href="/" class="btn btn-secondary">← 返回首页</a>
            </div>
        </div>
    </div>
    """)

    return render_template_string(html, projects=projects)

@app.route("/users")
def all_users():
    users = fetch_all("""
        SELECT u.user_id, u.username, u.email, u.created_at, s.plan_type,
               (SELECT balance_after FROM wallet_transactions WHERE user_id = u.
                ORDER BY created_at DESC LIMIT 1) as balance
        FROM users u
        JOIN subscriptions s ON s.user_id = u.user_id AND s.status = 'active'
        ORDER BY u.user_id
    """)

    html = BASE_HTML_TEMPLATE.replace("{% block content %}{% endblock %}", """
    <div class="card">
        <div class="card-header">
            <h2>👥 用户列表</h2>
            <p>系统所有注册用户</p>
        </div>
        <div class="card-body">
            <table>
                <thead>
                    <tr><th>ID</th><th>用户名</th><th>邮箱</th><th>套餐</th><th>
                </thead>
                <tbody>
```

```
                    {% for u in users %}
                    <tr>
                        <td>{{ u.user_id }}</td>
                        <td>{{ u.username }}</td>
                        <td>{{ u.email }}</td>
                        <td><span class="badge badge-primary">{{ u.plan_type | u
                        <td>${{ "%.2f"|format(u.balance or 0) }}</td>
                        <td>{{ u.created_at[:10] }}</td>
                        <td><a href="/user/{{ u.user_id }}/dashboard" class="btn
                    </tr>
                    {% endfor %}
                </tbody>
            </table>
            <div style="margin-top: 24px;">
                <a href="/" class="btn btn-secondary">← 返回首页</a>
            </div>
        </div>
    </div>
    """)

    return render_template_string(html, users=users)

# 运行应用
if __name__ == "__main__":
    app.run(host="127.0.0.1", port=5001, debug=False, use_reloader=False)
```

```
 * Serving Flask app '__main__'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment.
Use a production WSGI server instead.
 * Running on http://127.0.0.1:5001
Press CTRL+C to quit

 * Running on http://127.0.0.1:5001
Press CTRL+C to quit
127.0.0.1 - - [29/Apr/2026 19:03:07] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [29/Apr/2026 19:03:07] "GET /favicon.ico HTTP/1.1" 404 -
127.0.0.1 - - [29/Apr/2026 19:03:15] "GET /user/1/dashboard HTTP/1.1" 200 -
127.0.0.1 - - [29/Apr/2026 19:03:21] "GET /projects HTTP/1.1" 200 -
127.0.0.1 - - [29/Apr/2026 19:03:23] "GET /projects HTTP/1.1" 200 -
127.0.0.1 - - [29/Apr/2026 19:03:23] "GET /users HTTP/1.1" 200 -
127.0.0.1 - - [29/Apr/2026 19:03:26] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [29/Apr/2026 19:03:28] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [29/Apr/2026 19:03:30] "GET /users HTTP/1.1" 200 -
127.0.0.1 - - [29/Apr/2026 19:03:32] "GET /user/1/dashboard HTTP/1.1" 200 -
127.0.0.1 - - [29/Apr/2026 19:03:36] "GET /user/2/dashboard HTTP/1.1" 200 -
127.0.0.1 - - [29/Apr/2026 19:03:50] "GET /chat/3/messages HTTP/1.1" 200 -
127.0.0.1 - - [29/Apr/2026 19:04:08] "GET / HTTP/1.1" 200 -
```